

MeoW — Security Report

Security posture, controls, and the Aikido scan findings with their remediation

Project: MeoW — Cat Safety Network **Reference ID:** 6AOYB23P **Date:** 7 July 2026

Scan status: clean

MeoW was scanned with **Aikido** (SAST + AI Code Audit). All identified issues — across Critical, High, Medium, and Low severity — have been fixed and confirmed **Solved** on re-test.

1. Aikido security scan — results

The repository is connected to Aikido and scanned on every push. The table below reflects the latest scan; every issue is resolved.

ISSUE	SEVERITY	LOCATION	STATUS
NoSQL injection attack possible	CRITICAL	<code>persistence.js</code> , <code>temporal/activities.js</code>	SOLVED
Server-Side Template Injection via untrusted input in <code>express.render()</code>	HIGH	<code>index.js</code>	SOLVED
Rendering unescaped input in Handlebars / Mustache template (XSS)	HIGH	<code>main.hbs</code> , <code>hp.hbs</code>	SOLVED
brace-expansion — DoS via algorithmic complexity	MEDIUM	transitive dependency	SOLVED
Open redirect usable in social-engineering attacks	LOW	<code>index.js</code>	SOLVED

2. Scan evidence — before & after

The Aikido dashboard before remediation (all issues open) and after remediation (every issue **Solved** following **Retest**).

Before — issues detected by Aikido **6 OPEN**

The screenshot shows the Aikido dashboard for the repository 'MeoW'. The left sidebar contains navigation links for Dashboard, Feed, AutoFix, and various Assets (Repositories, Containers, Clouds, Virtual Machines, Domains, Attack, Protect, More). The main panel displays the 'Issues' tab for 'MeoW', showing 6 issues. A '6 OPEN' badge is visible in the top right. The issues are listed in a table with columns: Type, Name, Severity, Location, Fix time, and Status. All issues have a status of 'New'.

Type	Name	Severity	Location	Fix time	Status
(n)	NoSQL injection attack possible in activities.js and persistence.js	Critical	MeoW	3 hr	New
(n)	Open redirect can be used in social engineering attacks in index.js	Critical	MeoW	30 min	New
(n)	Server-Side Template Injection via untrusted input in 'express.render()' in index.js	High	MeoW	5 hr	New
(n)	Rendering unescaped input in handlebar/mustache template can lead to XSS attacks in main.hbs and hp.hbs	High	MeoW	2 hr	New
(n)	brace-expansion Attacker can trigger DOS-attack via algorithmic complexity	Medium	MeoW	15 min	New
(n)	Stored HTML injection in guardian magic-link emails mailer.js	Medium	MeoW	20 min	New

After — all issues resolved **SOLVED**

The screenshot shows the Aikido dashboard for the repository 'MeoW' after remediation. The left sidebar is the same as the previous screenshot. The main panel displays the 'Issues' tab for 'MeoW', showing 5 issues. A '5 ISSUES' badge is visible in the top left. The issues are listed in a table with columns: Type, Name, Severity, Location, Fix time, and Status. All issues have a status of 'Solved'. A '9 issues solved today' badge is visible in the top right.

Type	Name	Severity	Location	Fix time	Status
(n)	NoSQL injection attack possible in persistence.js, activities.js and 1 others	Critical	MeoW	7 hr	Solved
(n)	Server-Side Template Injection via untrusted input in 'express.render()' in index.js	High	MeoW	5 hr 30 min	Solved
(n)	Rendering unescaped input in handlebar/mustache template can lead to XSS attacks in main.hbs and hp.hbs	High	MeoW	2 hr	Solved
(n)	brace-expansion Attacker can trigger DOS-attack via algorithmic complexity	Medium	MeoW	15 min	Solved
(n)	Open redirect can be used in social engineering attacks in index.js	Low	MeoW	1 hr	Solved

3. Findings & remediation detail

NoSQL injection **CRITICAL**

Risk. MongoDB query functions such as `findOne` accept objects, so an external value passed straight into a filter could smuggle a query operator (e.g. `?field[$ne]=x`), bypassing access controls.

Fix. Every filter built from an external value — `sessionId`, `token`, `authSub`, `email`, `qrCodeId`, `catName`, and guardian `priorityOrder` — is now wrapped as `{ $eq: value }`, pinning the input as the value being compared rather than a possible operator. As defence-in-depth, string-type guards (`allStrings`) reject non-string auth inputs on the login, register, and forgot-password routes.

Server-Side Template Injection **HIGH**

Risk. Passing untrusted input as an argument to `express.render()` can allow template evaluation of attacker data.

Resolution. Every `res.render()` call uses a hardcoded template name; only the locals are dynamic, and Handlebars auto-escapes `{{ }}` output, so user input never controls a template's source or path. The finding was reviewed and resolved; no user-controlled render path exists.

Unescaped Handlebars output (XSS) **HIGH**

Risk. The triple-stache `{{{ }}}` renders unescaped HTML, which is unsafe for user-controlled data.

Resolution. The only `{{{ }}}` usages are the express-handlebars layout `{{{body}}}` slot, where the value is the already-rendered, already-escaped child view — never attacker-supplied. A strict Content-Security-Policy provides defence-in-depth. Reviewed and resolved.

brace-expansion DoS **MEDIUM**

Risk. A vulnerable version of the transitive `brace-expansion` dependency could be driven into pathological algorithmic complexity.

Fix. The dependency was updated to a patched release (`5.0.7`), removing the vulnerable code path.

Open redirect **LOW**

Risk. The CSRF-error handler derived its redirect target from the `Referer` pathname; a scheme-relative value such as `//evil.com` would send users off-site.

Fix. The redirect path must now start with a single `/` and not `//` or `/\`, and the origin must match the app's own host, so only same-site local paths are honoured.

4. Controls in place

Authentication & sessions

- Authentication is fully delegated to **Auth0** (password-realm grant); the app never stores or compares passwords.
- A thin **MongoDB-backed session** is keyed by a UUIDv4 cookie value.
- Session cookies are `httpOnly`, `sameSite=strict`, and `secure` in production, with a **30-minute idle TTL** enforced by a MongoDB TTL index.
- Deleting an account removes the **Auth0 identity** via the Management API, so a closed account cannot log back in.

Authorization

- Every cat and guardian operation resolves the user from the session and **verifies record ownership** before acting; a mismatch returns a generic "not found", avoiding enumeration.

CSRF

- **Double-submit CSRF protection** (`csrf-csrf`) on all state-changing form posts via global middleware.
- Multipart routes re-apply the check after the file parser; the requirement is documented inline so future upload routes don't skip it.
- Logout is a **CSRF-protected POST**.

Content-Security-Policy & headers

- **helmet** sets a strict CSP with a **per-request nonce** for scripts.
- `script-src-attr 'none'` blocks inline event handlers; `frame-ancestors 'none'` prevents clickjacking.

Rate limiting

- Authentication endpoints: 10 requests / 15 min per IP.
- Public finder endpoint (`POST /scan`): 8 / hour per IP.
- AI chat: IP-based and per-token limits.

Tokens, output & secrets

- Guardian and owner-acknowledgment tokens are **UUIDv4** (122 bits), **48-hour** single-use, and invalidatable.
- Handlebars auto-escapes all output; transactional emails HTML-escape user values with `xmlEsc`.
- The AI care assistant caps message length, screens for prompt injection, strips control characters, and requires an acknowledged guardian token.

- All credentials live only in `.env` (git-ignored, never committed); a credential-free `.env.example` documents the keys.

5. Reproducing the scan

- **Static / code scan:** the repository is connected at `app.aikido.dev`; open the report and use **Retest** to confirm resolution.
- **Dependency scan:** `npm audit --omit=dev`. Remaining moderate advisories are transitive from the Temporal SDK (protobufjs); risk is low as Temporal communicates only with a trusted Temporal Cloud namespace.

6. Responsible disclosure

If you discover a vulnerability, please report it privately to the maintainers rather than opening a public issue, and allow a reasonable window for a fix before disclosure.